on your PC against a ROM installation on your Android phone. Without a doubt, installing Windows is much friendlier than flashing ROMs. Since embedded systems are not normal PCs and the software written for them has to be correct and corruption free to the last bit, installing an update for them is a lot harder.

These security issues must be kept in mind right at the outset when designing the hardware. While software level updates can be made easier, there must be a limit to the frequency of updates due to various reasons such as high difficulty level and risk, legacy code, need to be always-up and stringent performance requirement. At the same time, the software layer of the embedded system must be built only after the hardware part is completed. A hardware change can bring in significant problems for software writers and can delay production or cause errors to creep in. This is another reason why components have to be carefully selected and organised in an attack-isolated fashion.

Availability of common time to all components

There are two possible interpretations of the phrase 'common time':

1 Common CPU Time: All components attached to an embedded system need to use the same CPU time. Assuming you're watching a video on your smartphone, your data connection and video decoder would be both putting some computation load on the processor. Also, other components such as Bluetooth and GPS antenna might be sending interrupts to the processor at the same time. Since you can't simply change a processor in an embedded system, the mechanisms to allow interrupts and (if needed) limit resource usage by various components need to be thought out when designing the system. This requirement is also referred to as 'CPU Time Sharing'.

2 Common Timestamp: If there's a need for one component of the system to interpret messages sent by another component periodically, it becomes necessary for both the components to read time from the same clock and calculate time differences. In case they use different clocks, the receiving component will reject the messages from the sending component if the sender's message has a timestamp which is from the future (i.e. the receiver component's clock is running behind the sender component's clock).

Interestingly, the common timestamp feature can aid in system security by saving from 'replay attacks' wherein the same message is sent more than once and can cause unwanted results on the receiving end.